

Desacoplamento e UI agnóstica

Relatório pós-remediação · 19 de agosto de 2026 · Escopo: aplicação WhatsApp Sender

RESULTADO	VALIDAÇÃO	RISCO RESIDUAL
8 de 8 achados tratados	32 testes + lint + tipos + build	Sincronização de opt-out offline permanece futura

1. Parecer executivo

A aplicação foi reorganizada para que as bordas HTTP, os casos de uso, as integrações concretas e a apresentação tenham responsabilidades explícitas. As rotas deixaram de acessar Prisma e WhatsApp diretamente; a integração WhatsApp foi decomposta; regras de domínio deixaram de conhecer ícones, classes CSS e formatação visual; e hooks passaram a receber feedback e confirmação por portas injetadas.

Os stores foram divididos por responsabilidade. Dados operacionais não são mais persistidos no navegador: o servidor permanece como fonte de verdade, enquanto o cliente mantém snapshots de sessão e apenas preferências locais versionadas. O acesso HTTP do frontend também foi centralizado.

2. Achados e ajustes aplicados

ID	Achado original	Remediação	Estado
A1	Rotas com regras e infraestrutura	Serviços de aplicação e factory; rotas agora validam, resolvem workspace e delegam.	CONCLUÍDO
A2	Domínio de engajamento acoplado à UI	Cálculos ficaram no domínio; badges, ícones e datas migraram para apresentação.	CONCLUÍDO
A3	Serviço WhatsApp monolítico	Gateway, gerenciador de conexão, handler de entrada e composition root separados.	CONCLUÍDO
A4	Store global com múltiplas responsabilidades	Stores de contatos, transmissão, preferências, filtros e simulação separados.	CONCLUÍDO
A5	Serviços dependentes de validadores HTTP e workspace implícito	Contratos de domínio próprios e workspace obrigatório nas chamadas de serviço.	CONCLUÍDO
A6	Hooks acoplados a Sonner, DOM e eventos globais	Portas de feedback/confirmação, refs React e stores tipados substituíram acoplamentos.	CONCLUÍDO
A7	Estado operacional persistido no browser	Contatos e campanhas passaram a usar o servidor como fonte de verdade.	CONCLUÍDO
A8	Chamadas fetch e erros inconsistentes	Todos os clientes de frontend usam requestJson e ApiRequestError.	CONCLUÍDO

3. Fronteiras arquiteturais resultantes

Entrada HTTP	src/app/api Autenticação, validação de entrada, workspace e serialização.
Aplicação	src/server/services Orquestração de casos de uso sem componentes visuais.
Domínio	src/domain e src/lib Contratos e regras reutilizáveis, sem Next.js ou bibliotecas de UI.
Portas e composição	src/server/ports e composition Interfaces externas e montagem das implementações.
Infraestrutura	src/infrastructure WhatsApp Web e outros detalhes concretos.
Apresentação	src/presentation, hooks e components Adapters visuais, estado de tela e composição React.
Estado cliente	src/stores Stores pequenos e orientados a uma responsabilidade.

4. Evidências de verificação

Verificação	Resultado
ESLint	Aprovado sem erros ou avisos
TypeScript (tsc --noEmit)	Aprovado
Vitest	8 arquivos, 32 testes aprovados
Testes de fronteira	5 invariantes arquiteturais aprovadas
Next.js production build	Compilação e geração de rotas concluídas
git diff --check	Sem erros de conteúdo; apenas avisos de normalização LF/CRLF

As invariantes automatizadas bloqueiam: rotas importando Prisma ou WhatsApp concreto; serviços de aplicação importando validadores HTTP; hooks importando Sonner; clientes chamando fetch fora do módulo HTTP compartilhado; e regras de engajamento incorporando dependências visuais.

5. Consentimento, opt-out e operação offline

O opt-out recebido pelo WhatsApp é capturado automaticamente enquanto o processo da aplicação está ativo e cada mudança é registrada na trilha de auditoria. A camada de entrada do WhatsApp agora depende de uma porta própria e recebe o workspace explicitamente, reduzindo o risco de mistura de contexto.

Limitação conhecida: se a mensagem chegar enquanto todo o processo estiver fechado, a captura não ocorre naquele instante. A implementação futura está documentada no README: sincronizar mensagens recentes após a reconexão, persistir um cursor idempotente e registrar na auditoria tentativas, origem da sincronização e falhas.

6. Riscos residuais e recomendações

1. Implementar a sincronização offline antes de depender do opt-out automático como único canal de conformidade.
2. Manter os testes de fronteira no pipeline de CI e exigir o comando `npm run check` antes de releases.
3. Em uma futura expansão multi-tenant, substituir o resolvedor local por identidade autenticada sem reintroduzir workspace implícito nos serviços.
4. Para escala horizontal, externalizar coordenação de fila e propriedade da sessão WhatsApp.

7. Conclusão

Os oito achados da auditoria foram tratados no código e protegidos por verificações automatizadas. A UI está substancialmente mais agnóstica, os casos de uso estão isolados das bordas de transporte e a integração WhatsApp possui pontos de extensão explícitos. O projeto está apto a evoluir as funcionalidades atuais sem depender de uma reescrita arquitetural.

Status final: **APROVADO COM UMA LIMITAÇÃO OPERACIONAL DOCUMENTADA** - sincronização de opt-out recebido durante indisponibilidade total do processo.